

Research Challenges on Service Technology Foundations

Dimka Karastoyanova

*IAAS, University of Stuttgart, Stuttgart, Germany
karastoyanova@iaas.uni-stuttgart.de*

Dragan Ivanovic,

*UPM, Madrid, Spain
idragan@clip.dia.fi.upm.es*

Zsolt Neméth

*Computer and Automation Research Institute
MTA SZTAKI, Budapest, Hungary
zsnemeth@sztaki.hu*

Manuel Carro

*UPM and IMDEA Software Institute, Madrid, Spain
mcarro@fi.upm.es*

Claudia Di Napoli, Maurizio Giordano

*Istituto di Cibernetica “E. Caianiello”, C.N.R.
Pozzuoli (Naples), Italy
[c.dinapoli, m.giordano]@cib.na.cnr.it*

Cesare Pautasso

*University of Lugano
Lugano, Switzerland
cesare.pautasso@usi.ch*

Abstract— This work gives an overview of the future research challenges on enabling technologies for service-based applications that have been identified in the network of excellence S-Cube. Service-based applications comprise three layers: business processes, service compositions and services and service infrastructures. The goal of this work is to present a roadmap for future research in technologies for software and system services.

Keywords—service-based applications; challenges; technologies; formal models; service compositions; business transactions; KPIs; service networks; monitoring and adaptation; nature-inspired approaches; chemical composition; liquid computing; Cloud computing.

I. INTRODUCTION

The advances in Service Oriented Computing (SOC) and Service Oriented Architectures (SOAs), as the latest development in application integration, have brought about major benefits for enterprises. The fast advances in these areas mainly driven by industry have led to a relative lack of systematically documented and integrated knowledge, in particular in the areas of software development, deployment and maintenance, as well as a very heterogeneous landscape of sometimes overlapping technologies and techniques that can be used for the development of software services and service oriented systems. In order to address this major problem, integration of research has been performed in a systematic manner by the researchers participating in the Network of Excellence on Software Services and Systems (S-Cube, <http://www.s-cube-network.eu/>) chartered and supported by the European Commission. The participating organizations belong to four different research communities, namely business process management (BPM), Software Engineering, SOC and Grid Computing.

Our findings in research and as a result of our attempt to organize research and expertise of four research communities motivated us to derive and capture the challenges we identify for future research in Service-based applications (SBAs).

The future research challenges presented here are only focusing on the categories of open issues in the technological underpinnings of SBAs. They have been derived in close collaboration and exchange with researcher groups in S-Cube who were working in the following areas (1) design principles and service engineering, (2) cross-layer monitoring and adaptation of SBAs, and (3) service quality assurance. For details on the challenges in future research identified in these areas the readers are referred to [11], [12], [13] and [14].

This paper is organized as follows: Section II introduces the major challenges we have identified for the future of Software System Development; sections III through VI describe the challenges in more details and present pointers to future research opportunities. We summarize our contributions and present conclusions in section VII.

II. CHALLENGES

As mentioned above, the challenges for future research are a result of the collaboration among researchers during their work on establishing enabling technologies for SBAs. We assumed the following SBA model: an SBA is a realization of a business process which is enacted using BPM technology, e.g. service choreographies (see Figure 1). The roles in a choreography are realized using orchestrations (our focus was on the service compositions approach). The individual tasks in a service orchestration are implemented by services, which form the lowest layer of the three layer architecture of an SBA, as shown in Figure 1. We identified three major concerns that cross-cutting the architecture of SBAs, namely (1) engineering and design principles for SBAs, (2) adaptation and monitoring of SBAs and (3) SBA quality definition, negotiation and assurance. This model allowed for defining a very flexible structure for organizing our research work, which also facilitated the interaction among researchers and fostered a systematic approach towards achieving the goals of the project.

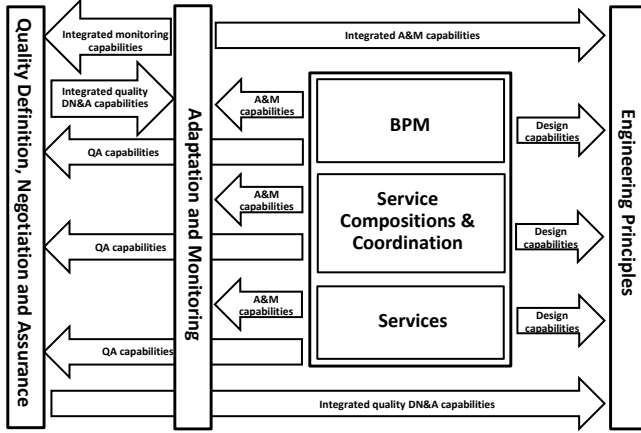


Figure 1. Functional Layers in the SBA model and cross-cutting concerns.

This paper presents the roadmap for enabling technologies for SBAs and comprises the following challenges:

1. Formal languages and composition approaches supporting information flow modelling and data-related quality analysis.
2. Adaptation patterns for service compositions based on quality attributes with emerging importance, such as greenness, trustworthiness, risk, value, etc.
3. Techniques for enabling and adapting agile service networks and business transactions.
4. Novel architectural styles and paradigms for handling large-scale, adaptable service-oriented systems. Here we have identified two potential directions for future research: chemical computing and liquid computing.
5. Techniques for leveraging and integrating social computing techniques and humans into adaptable business processes.
6. Exploiting cloud solutions (IaaS, PaaS, SaaS) in software and system services.

While we have identified all these challenges, we will present all but the last two challenges, since for them there are already existing strong communities with established roadmaps and detailed information on the open research questions is available. The identified challenges do not target explicitly only one of the layers of an SBA or only some of the cross-cutting concerns, but rather consider the dependencies between the SBA layers and take into account all the cross-cutting concerns.

III. FORMAL LANGUAGES AND COMPOSITION APPROACHES SUPPORTING INFORMATION FLOW MODELLING AND DATA-RELATED QUALITY ANALYSIS

Quality of Service (QoS) attributes – such as running time, resource consumption, or availability – are crucial for the practical usability and economic interest of service-based applications (SBAs). Those that rely on the service composition layer for realizing complex, maybe cross-organization tasks by arranging individual service

components are particularly challenging since such services are often provisioned and controlled by third parties.

Most widely used formalisms to describe and reason about services and service compositions (e.g., Petri Nets or pi-calculus) tend to focus on structures and dependencies derived from control concerns. However, in many applications data characteristics and data dependencies play an at least equally important role. In this section, we focus on data-related QoS and on logic representations of service compositions as formal models and identify some of the challenges and potential approaches to address them in this area.

The behaviour of a service (and, therefore, of a service composition) can significantly vary with the size and the content of its input data, leading to different QoS values. Therefore, taking into account a measure of data (such as its size, in some suitable metric) can significantly improve precision of QoS measurement and prediction. As shown in Figure 2, where dashed lines represent the prediction and shades the range of error around the prediction, data-insensitive, average-based QoS descriptions (top left) can be improved in two directions: using data-sensitive average predictions (top right) and range-based predictions (bottom left). Both can be combined (bottom right) in interval-based, data-sensitive predictions. Such data-sensitive QoS descriptions can be used to create QoS-sensitive compositions that rely on dynamic service matching to optimize QoS by adapting to the characteristics of the input data at hand [1]. Deriving the boundary functions can be based on computational cost analysis techniques applied to formal representations of service compositions (and the “atomic” component services), such as Horn clause programs which abstract away the operational details, but preserve enough composition semantics to allow inferring the complexity functions based on an abstraction of input data size [2], [3], [4].

On the other hand, data itself may drive the composition structure. A scenario would be the fragmentation of an orchestration for distributed enactment across different domains, where the information flow must respect privacy constraints. The task here is to infer data attributes for all activities and data items based on the (known) attributes of input data, the control structure of the orchestration, and data dependencies between its activities. While in simple and acyclic orchestrations the information flow can be trivially derived, it is an undecidable problem for general (Turing-complete) control constructs such as those found in real-world service orchestration languages. This challenge can be addressed by means of sharing analysis, a program analysis technique based on abstract interpretation. As in the previous case, this can be done on a Horn clause model of the orchestration [3], where data dependencies are represented by operations in first-order logic. This model can capture the effect of complex and nested data and control structures and sub-workflows. As shown in Figure 3, a user-defined characterization of input data is accepted in the form of an object-attribute context and internally

represented as a concept lattice (from Formal Concept Analysis).

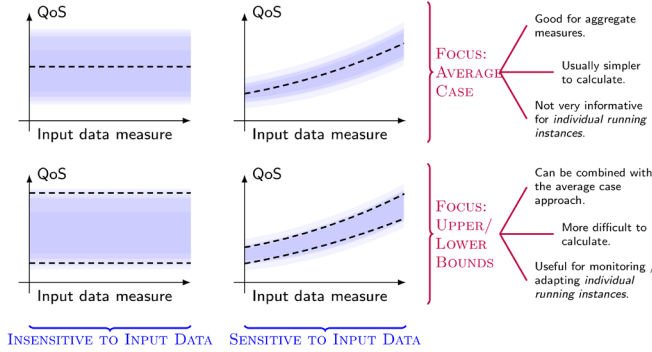


Figure 2. Data sensitivity and QoS precision.

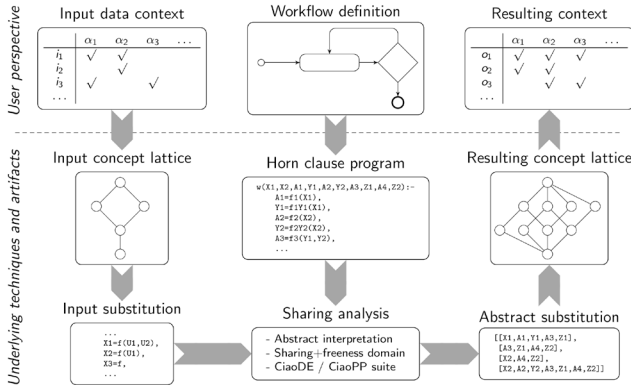


Figure 3. Sharing-based orchestration analysis.

The results of the sharing analysis are reinterpreted as lattices and concepts that assign the data attributes to all orchestration activities and control constructs, as well as to the intermediate and resulting data items [2].

IV. ADAPTATION PATTERNS FOR SERVICE COMPOSITIONS BASED ON QUALITY ATTRIBUTES

One of the major foci of the work on enabling technologies was on incorporating QoS and non-functional characteristics of SBAs in the overall model of applications and the corresponding mechanisms to adapt SBAs as a reaction to fluctuations in these characteristics.

For these purposes, in the scope of S-Cube we have developed a framework (see Figure 4) supporting this model and several approaches have been devised to enable different parts of the overall framework in terms of techniques and mechanisms.

The framework has been employed when developing models for monitoring of SBAs [17], to identify the different types of adaptation triggers – analysis results, on-line testing [14] and prediction of Key Performance Indicator (KPI) violations [15], and to identify the different types of analysis techniques. The framework has been realized in terms of generic architectures and technologies independent of application domains. The QoS characteristics we employed,

like response time, price, shipment time, average order duration, etc. are mostly relevant for business applications, as this was the domain of choice in S-Cube.

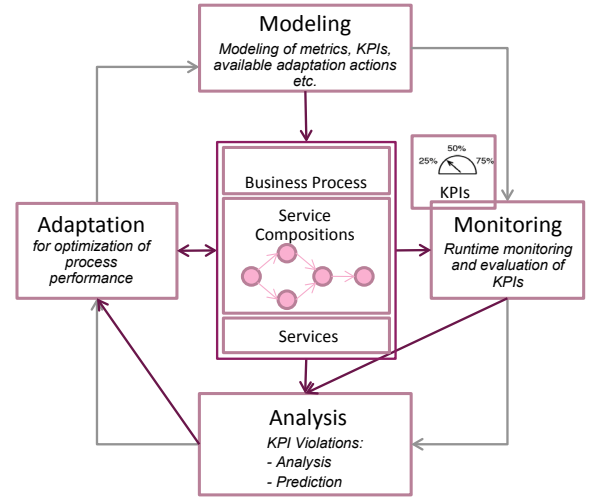


Figure 4. Framework for adaptable and QoS-aware SBAs.

With the latest developments in Cloud Computing and service provisioning, as well as having in mind global trends like GreenIT, we are convinced that other types of QoS characteristics need to be considered. Examples include the so-called Key Ecological Indicators (KEI), which are composite characteristics on the business process level, based on QoS on the service composition and service level [16], KPIs for trust, security, risk, sustainability and others. Future research work will have to define both models for such metrics and mechanisms for monitoring them. It may be necessary to identify new adaptation triggers and cross-layer adaptation mechanisms and position them within the S-Cube framework in order to address violations of these KPIs.

V. TECHNIQUES FOR ENABLING AGILE SERVICE NETWORKS AND BUSINESS TRANSACTIONS

Major focus of the research in BPM in recent years has been put on bridging the gap between the businesses and the IT support enabled through software services and systems. In S-Cube we introduced the concepts of business transactions (BTs) and agile service networks (SNs). They represent approaches towards simplifying the development of business processes using models and notations intuitive for business experts independent of the realization paradigm. BTs are a model to represent the interactions among partners in a business interaction that explicitly enforces the achievement of an agreed-upon business objective that applies to end-to-end business processes [20], [18]. A business transaction is also a model to describe the interactions of multiple companies in the form of exchanged goods, service level agreements, and common business goals [20]. The meta-model for business transactions, state model and a corresponding graphical notation have been defined in [20] and an extension to the graphical notation for BTs has been introduced for the purposes of monitoring [18] so that the

notation can be used to display the state of each of the actions in a BT.

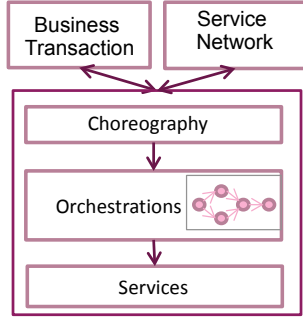


Figure 5. BTs and SNs as business views on SBAs.

Our approach for monitoring BTs is also based on the assumptions that BTs are realized in terms of choreographies that put together the orchestrations of partners/companies in a meaningful business coordination protocol (see Figure 5).

In order to be able to present the current state of a BT, the choreography state has to be monitored and mapped to the state model of BTs. Choreography monitoring has been enabled by our work in S-Cube, as reported in [12], [15], [17], and [18], and is basically using the monitoring information about orchestrations at each partner of the choreography, which in turn is based on the state of participating services (see Figure 5 and Figure 6). The need for adaptability imposes even more requirements on the realization and monitoring of BTs. In our work we have focused on enabling the monitoring of adaptable BTs by specifying a state mapping from split or merged orchestrations to BTs [18], as sketched in Figure 6.

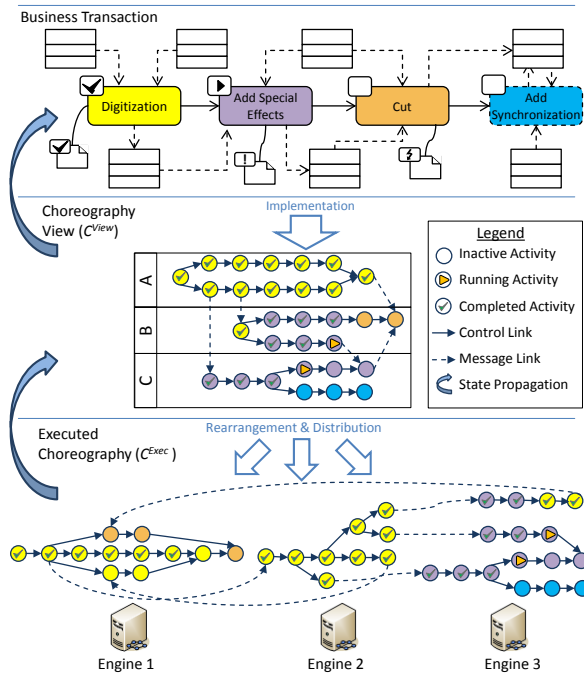


Figure 6 BT and original and rearranged choreography – a movie post-processing scenario [18].

We have however identified work for future research related to adaptability of BTs, namely: (a) there is a need to model the adaptability of BTs and provide the appropriate execution environment (Figure 6) and if necessary also extend the technology stack for Web Services [21], [18]; (b) for the design time phase of the BT life cycle verification techniques are required. Service networks present a different view on business processes [22], [19]. Service networks also model interactions among business partners in business processes in terms of services; however the focus in SNs is on the value of the SNs for the individual participants and the overall value of the SN. The work on SNs has been inspired by the observation that “globe-spanning networks of interdependent companies cooperate to provide value to their markets based on services (so-called *service value networks*)” [19]. In our research in S-Cube we focused on modelling, execution and monitoring of SNs, which enable the modelling and realization phases of the SVN lifecycle [19]. The meta-model for SNs has been presented in [23]. We used the BPM technology to realize SNs using choreographies and orchestrations (see Figure 5). There are still open research questions related to modelling and execution of service networks, as well as to the modelling of the value of SNs. Additional topics, related to these open issues, are the techniques for analysis of SNs (the third major phase in the SN life cycle). An approach based on state-transformation and views, similar to the one for BTs, is one option to be studied. However, other alternative approaches may also be needed.

The need to adapt both BTs and SNs is still not addressed. While we identify this as a challenge for future research in the area of adaptable SBAs, we are convinced that the solutions addressing this challenge have to be developed under consideration of the challenge to incorporate people and the knowledge about their social interactions into business processes (one of the major technological challenges identified in section II).

VI. NOVEL ARCHITECTURAL STYLES AND PARADIGMS FOR HANDLING LARGE-SCALE, ADAPTABLE SERVICE ORIENTED SYSTEMS

In our research work on the fundamentals of service-based applications and in collaboration with our associated partners, we also investigated unconventional approaches towards addressing the design, deployment and execution of adaptable SBAs. Indeed, the work on these approaches is at its beginning and they still need to be extended and evaluated with respect to their applicability. However, two approaches are very promising and focus on (a) improving the adaptability features of SBAs and (b) providing a better model and/or architecture for SBAs. We are convinced that it is of benefit to consider these approaches, and in particular to find out to what extent they can be combined with more established existing approaches. We introduce the chemical computing approach and the architectural style for liquid services in the next subsections.

A. Employing Nature-inspired Unconventional Paradigms for Service Infrastructure Adaptation

It is a widespread assumption that with the advent of Cloud Computing and the emerging IT platforms, Service Based Applications will be provided according to market-based criteria and as such they should be characterized as *Quality-aware Service Based Applications* (SBAs).

These applications are:

- built by composing third-party services available in an open and heterogeneous environment
- managed and operated in a completely decentralized way,
- required to satisfy end-to-end quality requirements,
- provided according to market-oriented approaches that regulate the demand and supply of services.

In a market-based scenario for the provision of Quality-aware SBAs, it will be necessary to furnish frameworks for the development and the execution of SBAs with support for flexibility, adaptation, and self-managing capabilities to better respond to sudden changes in the preferences of users as well as in the provision of quality factors specific of each component service with minimal human intervention. This dynamicity depends on market trends, and on other factors that cannot always be planned in advance. These aspects will become even more crucial with the emergence of infrastructures for the pervasive provisioning of digital services that need to interact with each other and to adapt to changes in the dynamic operational environment they operate in.

With traditional service-oriented approaches, self-adaptability can be integrated with additional tools and methodologies that may result in very complex and heavyweight systems.

Nature-inspired computing (NIC) [8] seems to be a promising approach to model self-adaptability features since they are inherent characteristics of natural systems. So, it is very likely that the adoption of NIC may unburden the complexity of such service infrastructures.

Nature-inspired computing is emerging as a way to model service-based systems as natural ecosystems so to reproduce their inherent autonomous and emergent (lifelike) behaviour in solving computing problems. In fact, the possibility to model and deploy services as autonomous entities “living” in an ecosystem of services, data sources, and devices, and to enforce autonomic behaviour as inherent properties of the ecosystem, rather than as peculiar characteristics of its entities has already been proposed by several initiatives [7]. Among the different nature-inspired approaches, chemical computing [9] may address some of the requirements of a framework for the development and execution of SBAs equipped with adaptability features.

In the S-Cube project, chemical computing was investigated as a programming paradigm to model the instantiation of SBAs in terms of a chemical process [10]: service offers, coming from a market of services, are modelled as chemical molecules characterized by the quality values they provide, and by other chemical parameters that drive their interactions, while service interactions are

modelled by chemical rules responsible for the system evolution. Chemical rules are characterized, by locality, lack of centralized control, evolutionary behaviour. The proposed chemical process allows on one hand to continuously process incoming offers as new molecules that may enter the chemical system at any time, as well as to change on the fly the aggregation patterns by inserting new chemical rules to select service components to instantiate an SBA (see Figure 7). The mechanism proved to be a viable approach to provide the adaptation features above mentioned.

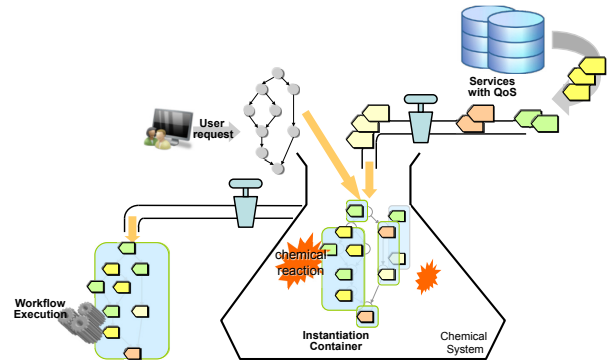


Figure 7. The chemical-based instantiation process.

Nevertheless, the characterizing features of chemical systems, such as the intrinsic non-determinism and locality of chemical computation, make it difficult to implement all the features of an SBA framework using only a chemical computing paradigm. So a hybrid approach mixing nature-inspired computing and more conventional computing models is more realistic to address the requirements of adaptable SBAs.

As future direction the potential of such hybrid chemical-inspired approach in supporting adaptation features in service infrastructures will have to be investigated to benefit from the intrinsic evolutionary nature of chemical system and, at the same time, from the consolidated technologies already available for managing and executing SBAs.

B. Liquid Architectural Style – Beyond SOA & RESTful

The liquid metaphor combines a number of properties of highly adaptable systems under a simple visual idea: the ability of liquids to seamlessly adapt their shape to the one of their container. Liquid Web services should feature the same ability to be deployed on heterogeneous execution infrastructures and make optimal use of the resources provided by each class of execution container to deliver their SBA with some quality guarantees. Given the large number of hardware infrastructures (shared memory multicore machines, distributed memory clusters) and software infrastructures (virtualized environments, IaaS, PaaS) that are available to deploy and execute services, it is very challenging to design a service which can achieve high performance, scalability, reliability across all possible execution environments. Indeed, services designed to exploit well a specific hardware/software environment will perform poorly (if at all) when redeployed across a different platform. Considering the emerging Cloud-based execution

infrastructures which make use of virtualization technology at different levels of abstraction, it becomes very difficult to estimate the properties of the targeted execution environment during the design of a service. Thus it is even more important to let the service adapt itself, dynamically to the execution environment in which it is deployed.

To address this challenge we have presented a novel architectural style [6], which can provide the necessary guidance both in terms of quality attributes to be guaranteed by an architecture and the design constraints that should be followed in order to achieve the desired quality attributes. Liquid Web services feature the following quality attributes: 1) Transparent Deployment 2) Predictable Scalability 3) Composability. The first quality is related to the liquid metaphor of adaptability, where the service is intended to be designed at a sufficiently high level of abstraction so that it can be automatically redeployed on very different execution environments. The second quality states that the service should take full advantage of whatever resources are available to deliver predictable performance, in terms of scalability. For example, if a multicore machine is available all cores should be exploited up to saturation to achieve linear scalability. If the possibility to offload some work to the Cloud is available, the service should take advantage of it only when the local resources are saturated. The third quality makes it explicit that liquid services are still meant to be used within service oriented architectures, and thus should reuse one another within larger service compositions. It should also be allowed to compose liquid services with “solid” services, which do not feature the first two qualities.

In the design process of liquid services we need to postpone making all design decisions concerning or depending on the deployment of the service until runtime, when they should be made automatically. To do so, we have selected a number of design constraints, which can provide the necessary guidance to the service designer. First, the internal architecture of services should be decomposed into fine-grained elements. The grain of each component should be as fine as possible to give maximum flexibility in the way components can be deployed, dynamically replicated and migrated. Stateless components should be explicitly distinguished from stateful ones. Second, the connectors between components should be underspecified. Assuming that services use asynchronous, message-based interactions for coordination, we do not need to specify which concrete communication mechanism will be used at run-time to carry the messages. Leaving the connectors underspecified at design-time helps to take advantage of the most efficient communication mechanism available at run-time (e.g., shared memory buffers on multicore machines vs. message queues on a distributed, share-nothing cluster). The third constraint concerns the specification of service interfaces, where the semantics of the possible interactions between services should be made explicit. The design should specify what the effect on the state of a service is as a consequence of processing a given message. Knowing whether a message would read or change the state of a component, or if it would not affect it, is very important to ensure the scalability of stateful services by means of replication.

VII. SUMMARY AND CONCLUSIONS

Our research in the area of enabling technologies for software services and systems has led to integrating the view on this area of four communities – BPM, SOC, Grid, Software Engineering. We have developed a homogeneous systematic framework of models, approaches and techniques needed to design, deploy, execute and adapt SBAs. Based on this framework and our research results we have identified and documented open issues for future research and organized them in several major challenges and presented them in this paper. We identified that, in addition to the research results of S-Cube, there are still remaining issues to address in enabling adaptable, QoS-aware SBAs, in particular regarding the formal methods for modelling service compositions with focus on data flows, the coherent cross-layer adaptation and adaptation mechanisms, the enactment of business transactions and service networks as two different ways to view business processes. We also identified potential promising approaches worth considering in combination with the research results we produced.

We conclude that the research in the field of enabling technologies is influenced by both academia and industry, which makes it a challenging and exciting task, in particular because of the speed industry can sustain in developing new technologies and because of the ambition of academia to develop the best scientific solution. This, on the other hand, also leads to a slower development of such technologies in comparison with the definition and development of design principles and methodologies on behalf of the software engineering community.

The opportunity S-Cube provided to its participants and associated parties to integrate research and exchange ideas brought about faster results, which consider paradigms, techniques and principles from different communities and research institutes. We claim that these results are with higher scientific merits than research performed in an isolated community.

ACKNOWLEDGMENT

The research leading to these results has received funding from the European Community's 7th Framework Programme under the Network of Excellence S-Cube - Grant Agreement no. 215483.

REFERENCES

- [1] D. Ivanovic, M. Carro and M. V. Hermenegildo, "Towards Data-Aware QoS-Driven Adaptation for Service Orchestrations," in *Proc. IEEE International Conference on Web Services (ICWS 2010)*, Miami, FL, USA, 2010.
- [2] D. Ivanovic, M. Carro and M. V. Hermenegildo, "Automated Attribute Inference in Complex Service Workflows Based on Sharing Analysis," in *Proc. 8th IEEE Conference on Services Computing SCC 2011*, Washington, D.C., 2011.
- [3] D. Ivanovic, M. Carro and M. V. Hermenegildo, "Automatic Fragment Identification in Workflows Based on Sharing Analysis," in *Service-Oriented Computing - ICSOC 2010, LNCS, Num. 6470*, San Francisco, CA, 2010.
- [4] D. Ivanovic, M. Carro and M. V. Hermenegildo, "An Initial Proposal for Data-Aware Resource Analysis of Orchestrations

- with Applications to Predictive Monitoring," in *International Workshops ICSOC/ServiceWave 2009, Revised Selected Papers*, LNCS, Num. 6275, Stockholm, Sweden, 2010.
- [5] Navas, J.; Mera, E.; P., Lopez-Garcia; Hermenegildo, Manuel V.; "User-Definable Resource Bounds Analysis for Logic Programs" in Proc. 23rd International Conference on Logic Programming (ICLP'07), LNCS 4670, Springer, 2007
 - [6] D. Bonetta, C. Pautasso, "An Architectural Style for Liquid Web Services", Proc. of the 9th Working IEEE/IFIP Conference on Software Architecture (WICSA 2011), Boulder, USA, June 2011
 - [7] M. Viroli and F. Zambonelli, "A biochemical approach to adaptive service ecosystems," *Journal of Information Science*, vol 180, issue 10, 2010, pp. 1876-1892, doi: 10.1016/j.ins.2009.11.021.
 - [8] L. Kari and G. Rozenberg. "The many facets of natural computing," *Communication of ACM*, vol. 51, issue 10, 2008, pp. 72-83. doi: 10.1145/1400181.1400200
 - [9] J.-P. Banâtre, P. Fradet, and Y. Radenac, "Principles of chemical programming," Proc. Fifth International Workshop on Rule-Based Programming (RULE'04), *Electronic Notes in Theoretical Computer Science*, vol. 124, issue 1, Elsevier, 2004, pp. 133-147, doi:10.1016/j.bbr.2011.03.031.
 - [10] C. Di Napoli, M. Giordano, Z. Németh, and N. Tonello, "Using chemical reactions to model service composition," Proc. of the Second International Workshop on Self-Organizing Architectures (SOAR'10), ACM, 2010, pp. 43-50, doi:10.1145/1809036.1809.
 - [11] E. Di Nitto et al., "Research challenges on service life-cycle and software engineering," in *Proceedings of the ICSE 2012 Workshop on European Software Services and Systems Research – Results and Challenges (S-Cube)*, 2012.
 - [12] Marconi et al., "Research challenges on multilayer and mixed-initiative monitoring and adaptation of service-oriented systems," in *Proceedings of the ICSE 2012 Workshop on European Software Services and Systems Research – Results and Challenges (S-Cube)*, 2012.
 - [13] A. Metzger et al., "Research challenges on online service quality prediction," in *Proceedings of the ICSE 2012 Workshop on European Software Services and Systems Research – Results and Challenges (S-Cube)*, 2012.
 - [14] Metzger et al., "Research Challenges on Adaptive Software and Services in the Future Internet" in *Proceedings of the ICSE 2012 Workshop on European Software Services and Systems Research – Results and Challenges (S-Cube)*, 2012.
 - [15] Wetzstein, Branimir; Zengin, Asli; Kazhamiakin, Raman; Marconi, Annapaola; Pistore, Marco; Karastoyanova, Dimka; Leymann, Frank: Preventing KPI Violations in Business Processes based on Decision Tree Learning and Proactive Runtime Adaptation. In: *Journal of Systems Integration*, January, 2012.
 - [16] Nowak, Leymann, Schumm, and Wetzstein. An Architecture and Methodology for a Four-Phased Approach to Green Business Process Reengineering. In *Proceedings of ICT-GLOW 2011*, Springer Verlag.
 - [17] Guinea, Sam; Kecskemeti, Gabor; Marconi, Annapaola; Wetzstein, Branimir: Multi-layered Monitoring and Adaptation. In: *Proceedings of ICSE 2011*, Cyprus, December 5-8, 2011.
 - [18] Wagner, Sebastian, Christoph Fehling, Dimka Karastoyanova, David Schumm, "State propagation-based Monitoring of Service-based Business Transactions". 2012. (submitted for publication at the SCC 2012).
 - [19] Bitsaki, Marina; Danylevych, Olha; van den Heuvel, Willem-Jan; Koutras, George; Leymann, Frank; Manciapoli, Michele; Nikolaou, Christos; Papazoglou, Mike: An Architecture for Managing the Lifecycle of Business Goals for Partners in a Service Network. In: *Proceedings of ServiceWave 2008*.
 - [20] Willem-Jan van den Heuvel, Michael P. Papazoglou, 2010. Toward Business Transaction Management in Smart Service Networks, *IEEE Internet Computing*, vol. 14, no. 4, p. 71-75.
 - [21] G. Decker, O. Kopp, F. Leymann, K. Pfitzner, and M. Weske, Modeling Service Choreographies using BPMN and BPEL4Chor, in *CAiSE2008*. Springer, 2008.
 - [22] N. Caswell, S. Feldman, C. Nikolaou, J. Sairamesh, M. Bitsaki, G. D. Koutras and G. Iacovidis: Estimating Value in Service Systems – A theory and an example. In: *IBM Systems Journal*, Volume 47, No 1, 2008.
 - [23] Danylevych, Olha; Karastoyanova, Dimka; Leymann, Frank: Service Networks Modelling: An SOA & BPM Standpoint. In: *Journal of Universal Computer Science (J.UCS)*. Vol. 16(13), 2010.